

ChatGPT

Upgrade

This video is essentially a **complete System Design Interview Guide for Caching**. Here's a condensed interview-focused version with diagrams and key takeaways.

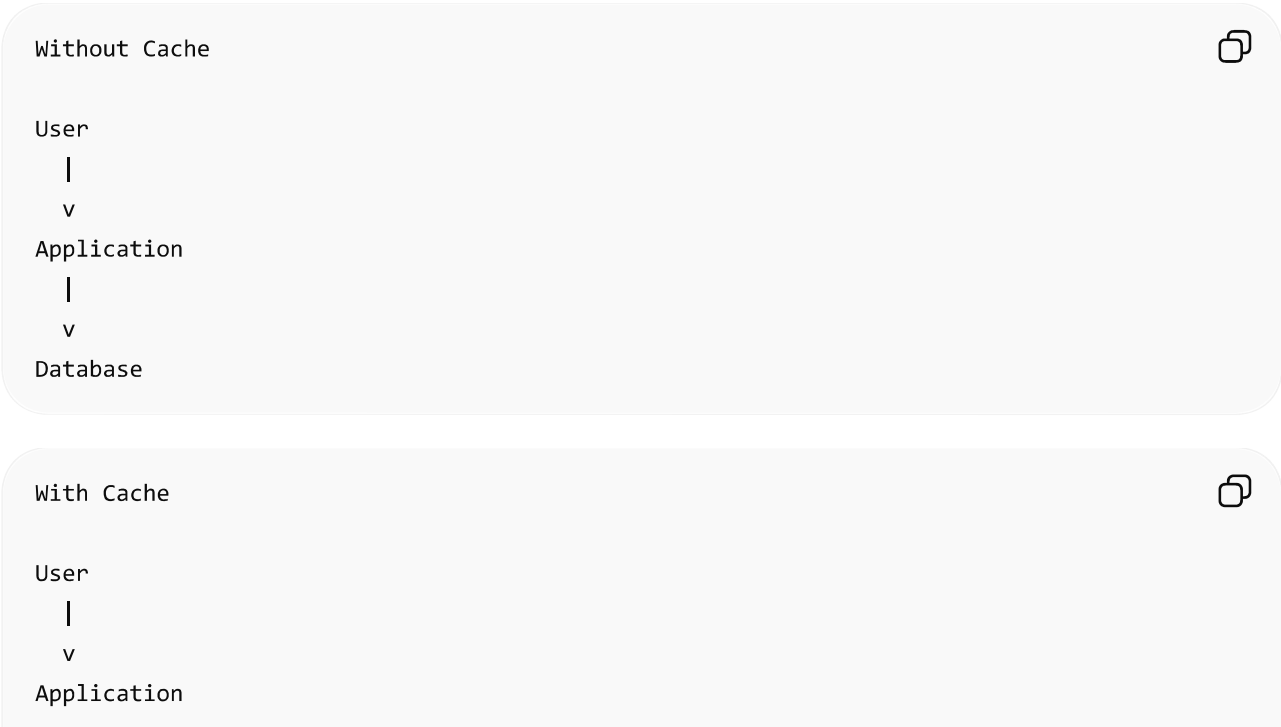
1. What is Caching?

A cache stores frequently accessed data in a faster storage layer (usually RAM) to avoid expensive database queries.

Why?

Storage	Latency
RAM (Cache)	~100 ns
SSD (DB)	~1 ms

RAM is roughly **10,000x faster** than disk.

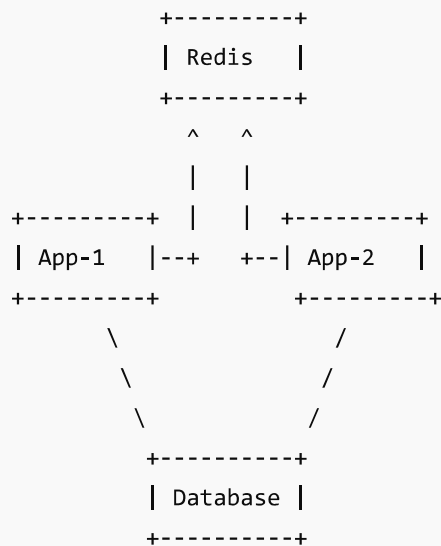


```
|  
v  
Cache (Redis)  
|  
| Miss  
v  
Database
```

2. Where Can We Cache?

A. External Cache (Most Common)

Using Redis/Memcached.



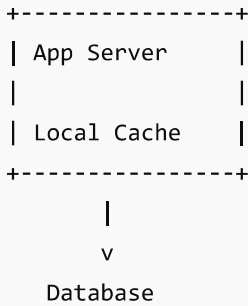
Pros

- ✓ Shared by all servers
- ✓ Scales well
- ✓ Most common interview answer

Examples

- Redis
- Memcached

B. In-Memory Application Cache



Pros

- ✓ Fastest

No network hop

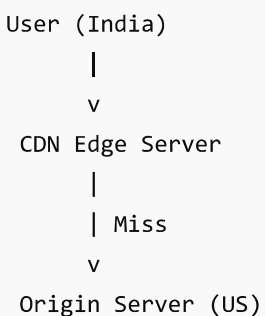
Cons

- ✗ Not shared across servers
- ✗ Cache inconsistency

Use Cases

- Configurations
- Lookup tables
- Feature flags

C. CDN Cache



Used For

- Images

- Videos
- Static Assets

Benefit

Instead of:

India -> USA = 300ms



You get:

India -> Nearby CDN = 20ms



Popular CDNs:

- [Cloudflare](#)
- [Akamai Technologies](#)
- [Amazon Web Services](#)

D. Client Side Cache

Browser

|

Local Storage

Memory Cache

HTTP Cache



Examples

- Angular service caching
- Browser cache
- Mobile offline mode

3. Cache Architectures

Cache Aside (Most Important)

Read Flow

1. Check Cache
2. Cache Hit -> Return
3. Cache Miss -> Query DB
4. Save to Cache
5. Return Response



Diagram:

```
Request
|
v
Cache
|
Hit? ----- Yes -----> Return
|
No
|
v
Database
|
v
Update Cache
|
v
Return
```



Interview Default

If interviewer asks:

How will you add caching?

Answer:

We'll use Redis with Cache Aside pattern.

Write Through

Application

|

v

Cache

|



v
Database

Write completes only after both succeed.

Pros

Fresh data.

Cons

- Slow writes
- Dual write problem

Write Behind (Write Back)

Application

|

v

Cache

|

| Async

v

Database



Pros

Very fast writes.

Cons

Data loss possible if cache crashes.

Use Cases

- Analytics
- Metrics

Read Through

App

|



```
v
Cache
|
| Miss
v
Database
```

Cache itself fetches from DB.

CDNs often work like this.

4. Cache Eviction Policies

Cache memory is limited.

Need to remove data.

LRU (Least Recently Used)

Remove data not used recently.

```
Oldest Accessed Item
  ↓
Evicted
```



Most Common Interview Answer

LFU (Least Frequently Used)

```
Item A -> 100 hits
Item B -> 2 hits
```



Evict B.

Best when traffic is highly skewed.

FIFO

1st inserted



removed



Simple but uncommon.

TTL (Time To Live)

User Feed

TTL = 60 sec



After 60 sec:

Automatically Expired



Very common in Redis.

5. Cache Stampede (Thundering Herd)

Most common deep-dive question.

Problem

Homepage Cache

TTL = 60 sec

100,000 users/sec



When cache expires:

100,000 Requests



Database



Database explodes.

Solution 1: Request Coalescing

Request-1 --> DB



Request-2 --> Wait

Request-3 --> Wait

Request-4 --> Wait

Only one request rebuilds cache.

Solution 2: Cache Warming

Before expiry:

TTL = 60



Refresh at 55 sec

Cache never expires.

Cron Job

|

v

Refresh Popular Keys



6. Cache Consistency

Problem

User changes profile picture.

Database:

Profile Pic = New



Cache:

Profile Pic = Old



Users still see old image.

Solution 1: Invalidate on Write

Update DB
|
Delete Cache Key



Next read fetches fresh data.

Solution 2: Short TTL

TTL = 30 sec



Accept temporary staleness.

Solution 3: Eventual Consistency

Accept stale data.

Good for:

- Feeds
 - Analytics
 - Metrics
-

7. Hot Keys

A single cache key gets enormous traffic.

Example:

Celebrity Profile



Millions of requests hit one Redis node.

Taylor Swift
|



v
Redis Node

Node becomes bottleneck.

Solution 1: Replicate Hot Keys

Taylor Swift
|
+---+---+
| | |
v v v
Redis1 Redis2 Redis3



Load balanced.

Solution 2: Local App Cache

App Server
|
Local Memory
|
Redis



Avoid Redis calls for extremely hot data.

8. Angular Examples of Caching

Based on your Angular background:

RxJS

<> TypeScript

```
http.get('/users')  
  .pipe(  
    shareReplay(1)  
  );
```



Avoid duplicate API calls.

NgRx Selector Memoization

</> TypeScript



```
createSelector(...)
```

Computed once.

Reused many times.

Signals

</> TypeScript



```
const total = computed(() => ...)
```

Result cached automatically.

HTTP Interceptor Cache

</> TypeScript



```
Request  
|  
Interceptor  
|  
Cache ?
```

Avoid repeated backend calls.

9. Interview Answer Template

Whenever interviewer asks:

How would you improve performance?

Use this structure:

Step 1

Identify bottleneck

Read-heavy workload



Step 2

Choose cache

Redis



Step 3

Choose strategy

Cache Aside



Step 4

Choose eviction

LRU + TTL



Step 5

Discuss risks

Cache Stampede
Cache Consistency
Hot Keys



30-Second Senior Engineer Answer

"Since this system is read-heavy, I'd introduce Redis as an external cache using the Cache-Aside pattern. Reads first check Redis; on a miss we fetch from the database and populate the cache. I'd use TTL-based expiration with LRU eviction. To prevent cache stampedes, I'd use request coalescing and cache warming for hot keys. For consistency,

I'd invalidate cache entries on writes and accept eventual consistency where appropriate."

This answer alone covers about 80–90% of what interviewers expect in a caching discussion.

       Sources